

ServiceNow Incident Client Scripts & UI Policies

Project Reference: SWTID-2026-8393 | Platform: ServiceNow System UI

1. Project Introduction & Meta Details

Overview

PROJECT CONTEXT

Project Title: Implement Client Script & UI Policy (Incident)

Target Table: Incident [incident]

Scope: Global Application Scope

TEAM LEADERSHIP

Team ID: SWTID-2026-8393

Team Leader: PALANI PRABA D

Contact Email: palanipraba15@gmail.com

PROJECT TEAM COMPOSITION

Member Name	Email Address	Assigned Role
NAVMITHA AFRIN S	navmithaaftrin@gmail.com	ServiceNow Developer
JAYAVARTHINI T	jayavarthinitheivendran@gmail.com	Client Scripting Specialist
SAHANA PARVEEN H	parveensahana821@gmail.com	Quality Assurance & Testing
ABINAYA R	abinaya8648@gmail.com	Technical Documentation Lead

2. Executive Overview & Problem Statement

Objectives

Problem Statement: Incident records require rigorous data entry controls and real-time field validation to ensure operational data consistency across IT Service Management workflows. Manual checking of incident inputs frequently causes incomplete records, inaccurate severity tagging, and unverified status updates.

Project Core Objectives:

- Implement dynamic UI Policies to enforce mandatory field inputs and control field read-only states based on Impact level.
- Automate real-time setting of the Urgency attribute when High Impact incidents are reported.
- Validate Incident record completion prior to form submission, preventing orphaned high-severity issues.
- Restrict unauthorized or unintended State changes directly from Incident list views.

3. High-Level Architecture

Architecture

The solution leverages native ServiceNow platform architecture without external framework dependencies:

- Presentation Layer:** ServiceNow Client Framework utilizing `g_form` and browser event listeners.
- Logic Layer:** ServiceNow Client Scripting Engine evaluating `onChange`, `onSubmit`, and `onCellEdit` events.
- Data Layer:** ServiceNow Relational Database enforcing data integrity on table `incident`.

4. Task Implementations & Configurations

Task 1 & Task 2

Task 1: Create UI Policy on Incident

A global UI Policy named `High Impact Control` was configured on the `incident` table. The condition evaluates if `Impact is 1 - High`. On evaluation, the policy enforces rules dynamically on load and when form values change, with automatic reversal enabled when false.



UI Policy - High Impact Control

Figure 1.1: UI Policy Creation Form - "High Impact Control" with condition `Impact = 1 - High`.

Task 2: UI Policy Action Configuration (Urgency & Assignment Group)

UI Policy Actions were attached to `High Impact Control` to lock the `Urgency` field as **Read-Only = True** and demand an `Assignment group` as **Mandatory = True** when high impact is detected.

5. Client Scripts Implementation

Task 3 & Task 4

Task 3: onChange Client Script – Auto Set Urgency

An onChange script responds to adjustments in the Impact field. When set to 1 - High, it programmatically updates Urgency to 1 - High and alerts the IT operator.

```
function onChange(control, oldValue, newValue, isLoading) { if (isLoading || newValue == '') { return; } if (newValue == '1') { g_form.setValue('urgency', '1'); g_form.addInfoMessage('Urgency set to High for High impact incident.')} }
```



onChange Client Script Configuration

Figure 3.1: Client Script definition for onChange execution on field Impact.

Task 4: onSubmit Client Script – Mandatory Validation

An onSubmit script validates whether Assigned To is populated prior to saving any incident with High impact.

6. List Governance Scripting

Task 5

Task 5: Create onCellEdit Client Script

To preserve process compliance, direct modification of the Incident State from list grids is disabled using an onCellEdit Client Script. Users are required to open the record to change state transitions.

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) { alert('State cannot be updated using list editing. Please open the Incident.');
```



onCellEdit Client Script Configuration

Figure 5.1: Configuration of onCellEdit script on State field preventing inline grid edits.

7. Quality Assurance & Testing Execution

Testing

Activity 1: Mandatory Field Enforcement Test

Validation confirmed that submitting a High Impact incident without an assigned user prompts an inline error banner and blocks record creation.



Validation Error Enforcement

Figure 6.1: Submission blocked displaying error "Assigned To is mandatory for High impact incidents."

Activity 2: Successful Record Save Verification

Populating Assigned To allows successful record save while keeping Urgency locked and mandatory rules active.



Successful Record Save

Figure 6.2: Successful save confirmation of INC0010005 with all mandatory rules satisfied.

8. List Blocking Test & System Verification

[Verification](#)

List Edit Blocking Modal Alert

Figure 7.1: Modal popup alert blocking list edit attempts on State column.

9. Comprehensive Test Traceability Matrix

[QA Summary](#)

Test ID	Scenario Description	Expected Behavior	Status
TC-01	Select Impact = High	Urgency set to High, message shown, Assignment group mandatory	PASSED
TC-02	Submit High Impact without Assigned To	Submission aborted; validation message shown	PASSED
TC-03	Attempt list edit on State field	Alert dialogue interrupts change; callback set to false	PASSED

10. Known Issues & Future Enhancements

[Roadmap](#)

Known Issues: Multiple UI Policies sharing identical order numbers (e.g., Order 100) on field Urgency trigger minor console execution warnings regarding unpredictable evaluation order.

Future Enhancements:

- Migrate legacy Client Scripts to Next Experience / Workspace client scripts for Service Operations Workspace compatibility.
- Integrate VIP caller logic to automatically boost Priority to Critical regardless of impact.